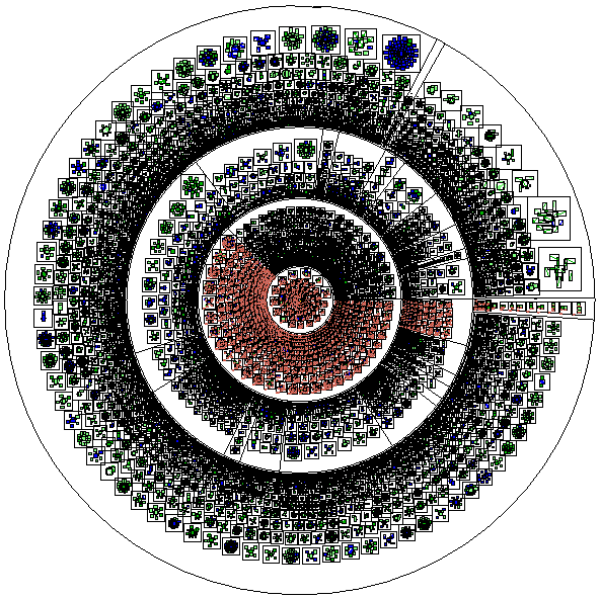
El kernel de Linux

Introducción

En este módulo hablaremos sobre el kernel de Linux y los módulos del kernel, haciendo hincapié en su compilación, instalación y mantenimiento.



Kernel y módulos del kernel

Como ya sabrá, el **kernel** es el software que sirve de puente entre el hardware y el resto de funciones del sistema operativo, que maneja el sistema de ficheros, el acceso a la red y otras tareas de bajo nivel que son críticas para el rendimiento.

El kernel está formado por un **fichero principal**, que

1º

Normalmente reside en `/boot` (o en el directorio raíz) y se denomina (generalmente) `vmlinuz-a.b.c-d`.

2º

Es el fichero que se indica al Gestor de arranque (bootloader) para que cargue el sistema operativo cuando el equipo arranca.

3º

Bastaría con indicar en el gestor de arranque un kernel diferente y reiniciar el equipo para cambiar de kernel si varios están instalados.

4º

Normalmente se le añade al nombre de fichero el número de versión, de manera que se pueden almacenar varios kernel aunque sólo uno de ellos esté operativo.

Además de este fichero, se instalan varios **módulos de kernel**, que normalmente se encuentran en `/lib/modules`

1º

Un módulo del kernel es un archivo que contiene código objeto que puede extender al kernel.

2º

Son generalmente utilizados para brindar soporte a nuevos dispositivos de hardware o nuevos sistemas de ficheros, así como para agregar llamadas al sistema.

3º

Cuando la funcionalidad provista por un módulo del núcleo deja de ser requerida, normalmente éste puede ser descargado, liberando su memoria.

4º

Un ejemplo típico de módulo cargable son los Controladores de Dispositivo (drivers).

Descarga e instalación del kernel

El kernel de Linux es común a todas las Distribuciones , y se descarga desde la página web www.kernel.org. Ahí están disponibles todas las versiones como *tarballs* de código fuente, que se compilan e instalan como cualquier otro software. Tal vez es necesario reconfigurar el boot loader para que localice el nuevo fichero de arranque del sistema operativo, y siempre es necesario reiniciar el sistema para que se apliquen las actualizaciones de kernel.

También es posible descargar actualizaciones (patch) del kernel actual a la versión inmediatamente siguiente, y aplicarlos con el comando `patch` al código fuente del kernel actual desde el directorio de dicho kernel.

```
#gzip -cd ../patch-version.gz | patch -pl
```

A continuación, se compila e instala el kernel actualizado como cualquier otro kernel. La descarga del parche siempre será más rápida que la de un kernel completo.

Importante:

Tenga cuidado siempre de aplicar sólo parches correspondientes a la versión actual de kernel, y no a otra. Para conocer cuál e la versión del kernel actual, se utiliza el comando

```
#uname -r
```

Versiones del Kernel

La versión viene indicada por una cadena de 4 número (`a.b.c-d`) en el nombre del fichero de kernel. El tercer número identifica una nueva "release", con nuevas funcionalidades, mientras que cambios en el último número indican simplemente que se han solucionado algunos errores (bugs). Las versiones del kernel se dividen en varias categorías:

Mainline

El árbol principal de actualizaciones del kernel es mantenido por Linus Torvalds. Las versiones mainline corresponden al desarrollo de nuevas funcionalidades, y se liberan cada 2-3 meses.

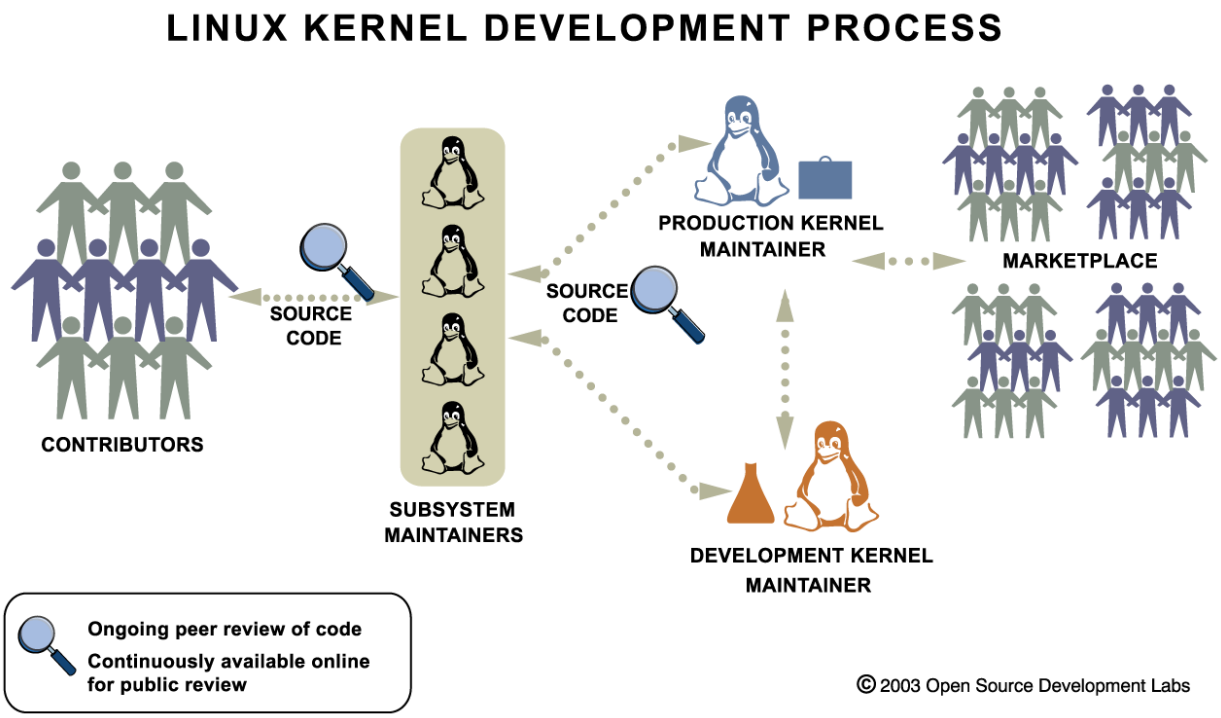
Prepatch

Corresponden a pre-lanzamientos de versiones mainline dirigidas a desarrolladores y entusiastas de Linux. No es aconsejable instalarlas en un sistema de producción, y su finalidad es la de que las nuevas funciones sean probadas por la comunidad antes de ser incluidas en una versión estable. Las versiones *prepatch* también son mantenidas y divulgadas por Linus Torvalds.

Estable
Después de una versión *mainline* se libera, se la considera "estable". Las correcciones de errores para un núcleo estable son portadas desde el árbol principal mantenido por Linus Torvalds y asignadas a otro desarrollador. Por lo general hay sólo unas pocas versiones del kernel de corrección de errores hasta que el núcleo principal siguiente está disponible - a menos que se designe un kernel de mantenimiento a largo plazo (longterm).

Longterm
Desde el árbol principal de actualizaciones se abren otras líneas de mantenimiento de versiones con el fin de proporcionar soporte a quienes no actualizan a versiones más nuevas. Solamente se aplican las correcciones más importante a esos núcleos y no suelen ser muy frecuentes, sobre todo para los árboles más viejos. Hay un desarrollador (maintainer) encargado de cada línea de mantenimiento que sale de la línea principal.

Gráficamente, se puede explicar de la siguiente manera:



En la siguiente tabla (extraída de kernel.org) se muestran cuáles son las líneas Longterm abiertas en el momento de editar este curso, quién es su desarrollador (maintainer) encargado, y cuál es la fecha prevista de finalización del mantenimiento (EOL, end-of-line):

Versión	Desarrollador	Fecha EOL
3.4	Greg Kroah-Hartman	Octubre 2014
3.2	Ben Hutchings	2016
3.0	Greg Kroah-Hartman	Octubre 2013
2.6.34	Paul Gortmaker	Mediados 2013
2.6.32	Willy Tarreau	Mediados 2014

El código fuente del kernel instalado en nuestro sistema se encuentra normalmente en `/usr/src/linux`. Es recomendable conservarlo este directorio incluso después de haber compilado e instalado el kernel, ya que contiene un **directorio con la documentación** asociada a dicho kernel, que nos puede ser de utilidad a la hora de necesitar un nuevo driver, por ejemplo. También es posible que instalemos software a partir de sus fuentes, que utilice alguna de las librerías declaradas en el kernel, y serían necesarias para poder compilarlo.

Anotación:

En la práctica, cada distribución de Linux pone a disposición actualizaciones propias del kernel, que no se obtienen desde *kernel.org*, sino que se distribuyen como cualquier otra actualización del sistema operativo. Estas actualizaciones son independientes a las mencionadas anteriormente, y por lo general se instalan a partir de paquetes (`yum`, `rpm`...) denominados `linux-source, kernel-devel, linux-headers, kernel-headers` (para las librerías), etc. En este caso, la actualización del kernel se hace de la misma manera que el resto de actualizaciones del sistema operativo, o paquetes de otras aplicaciones.

Configuración del Kernel

Para una correcta configuración del kernel, es necesario conocer nuestro hardware a fondo. El kernel incluye drivers para el hardware que van en función del chipset de nuestra máquina. Si se utiliza un kernel de una de las distribuciones de Linux, es más que probable que no sea necesaria ninguna configuración (estarán cargados los módulos kernel principales, se usen o no). Sin embargo, si se parte desde un kernel, una configuración incorrecta del hardware podría hacer que éste no arrancara. Hay una serie de comandos y ficheros que nos ayudan a identificar nuestro hardware:

lspci y lsusb

`lspci` y **`lsusb`** nos proporcionan una lista de los dispositivos PCI y USB (respectivamente) conectados en el sistema.

lsmod

Con **`lsmod`** obtenemos la lista de los módulos del kernel que están cargados actualmente. El problema es que esta lista no muestra los drivers que se cargan directamente en el kernel, y además el nombre dado al driver puede no ser fácilmente asociado con el dispositivo al que corresponde.
fichero .config

Se puede utilizar el fichero de configuración del kernel (**`.config`**) como modelo para conocer los drivers que vamos a necesitar.

Importante:

Se debe de prestar atención especial a los drivers de la controladora de discos, el adaptador de red, el controlador de video y el controlador USB, ya que son los que pueden dar lugar a que el sistema no arranque. Hay otros dispositivos (teclados, ratones, modems, discos ópticos, etc) que rara vez necesitan drivers específicos.

Al igual que con el software, como vimos en la unidad anterior, el primer paso es la compilación del código fuente con el comando **`make`**. Este comando tiene diferentes opciones

1º
Eliminar ficheros temporales de configuraciones anteriores (**`make mrproper`**)

2º
Crear un fichero de configuración con valores por defecto (**`make defconfig`**)

3º
Utilizar un fichero de configuración que ya se tuviera de antes (**`make oldconfig`**)

4º
Obtener un kernel que compile tantos módulos como le sea posible (**`make -allmodconfig)`**

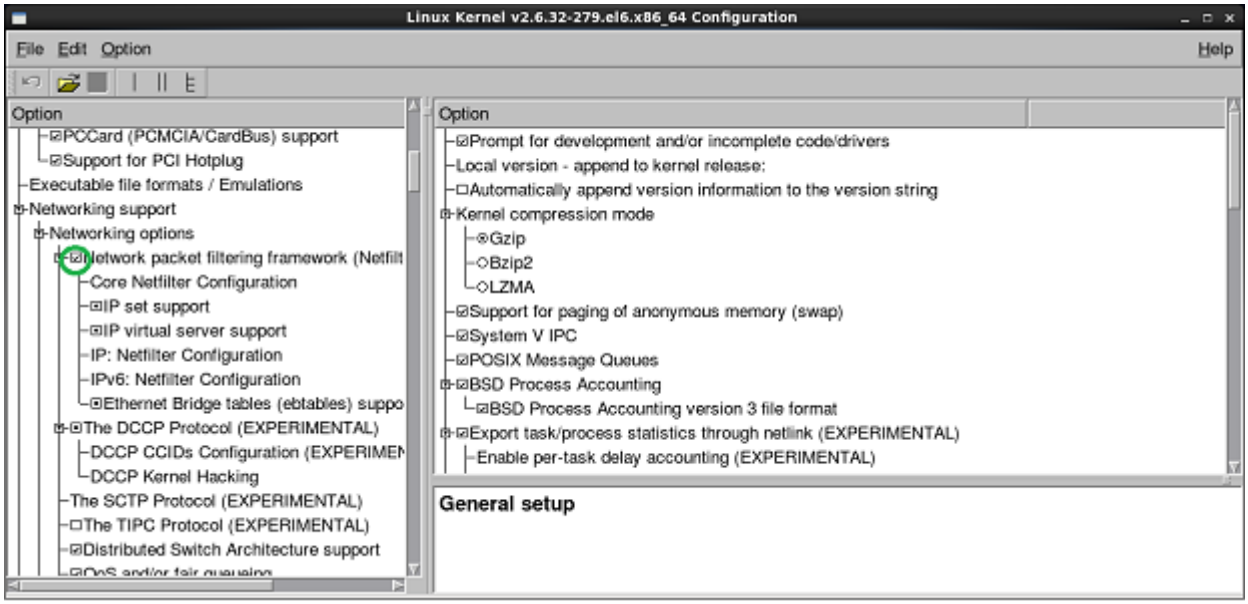
5º
Ver más opciones (**`make help`**). También puede consultar la documentación `/usr/src/linux/README`

6º
Configurar los elementos del kernel usando un interfaz:

- Basado en texto (**`make config`**)
- Basado en un menú (**`make menuconfig`**)
- Basado en una interfaz gráfica (**`make xconfig`** **0** **`make gconfig`**).

Estableciendo las opciones de configuración

En la siguiente imagen se muestra el interfaz correspondiente al comando `make xconfig`. Se pueden ver todas las opciones de configuración organizadas de manera jerárquica. Las categorías principales se localizan a la izquierda, y las subcategorías se van mostrando en el marco de la derecha. También se acompaña de pequeñas explicaciones de a qué corresponden las opciones. Una marca de validación indica que la opción se compilará directamente en el kernel, mientras que un punto indica que la opción será compilada como un módulo. También se visualiza para la opción seleccionada, una descripción detallada de su funcionalidad.



Importante:

Para poder compilar un núcleo, necesitará instalar los siguientes paquetes:
- **`yum groupinstall "Development Tools"`**: Esto asegurará que tenga todas las herramientas requeridas para la compilación.
- **`yum install ncurses-devel`** : Esto se requiere para hacer que *`make xconfig`* y *`make gconfig`* se ejecuten correctamente.
- **`yum install qt3-devel`** y **`yum install libxi-devel`** : Esto solamente es necesario si desea usar *`make xconfig`*.

Además, deberá contar con el árbol completo de fuentes del núcleo.

Compilación del Kernel

Una vez establecida la configuración, el siguiente paso es compilar el kernel y preparar al sistema para su utilización. Posteriormente, el kernel generado debe copiarse al directorio `/boot`, instalar los módulos del kernel, preparar un disco inicial de RAM y modificar la configuración de GRUB para que apunte hacia el nuevo kernel. Es posible usar la `make install` para automatizar alguna de estas tareas, pero nosotros las veremos paso a paso manualmente.

1. Generar el kernel

Para generar el kernel, simplemente ejecute el comando `make` sobre el directorio donde tiene las fuentes del kernel y el fichero de configuración generado en el paso anterior. Si su sistema dispone de todas las herramientas necesarias para la compilación y el fichero de configuración no contiene errores, el procedimiento llevará unos minutos en terminar. En todo caso, se irán visualizando mensajes de información o de error si ha habido algún problema.

2. Instalar el kernel

El kernel compilado satisfactoriamente puede ser "instalado", o simplemente copiado a su localización correcta, dentro del directorio `/boot`. Por lo general el nombre que se le suele dar al kernel cuando éste se ha obtenido de una compilación es `bzImage`, más la versión, para que puedan convivir varias versiones en el mismo directorio. Este nombre se usará después para reconfigurar GRUB. En algunas plataformas, el kernel se sitúa dentro del directorio `arch/x86/boot` o bien `arch/x64/boot`, según corresponda.

3. Instalar los módulos del kernel

Como `root` se lanza el comando `make modules_install`, que creará un nuevo subdirectorio bajo `/lib/modules` con el número de versión del kernel y copiará allí los módulos.

4. Preparar el disco inicial de RAM

Un disco inicial de RAM es una colección de modulos críticos del kernel y de utilidades de sistema que el bootloader lee del disco y pasa al kernel en el momento del arranque. Si el kernel ya incluye todos los drivers que se necesitan para arrancar dentro de su fichero inicial, este paso sería innecesario. Cada distribución utiliza una utilidad diferente para la generación del disco inicial de RAM, siendo los más comunes `mkinitrd` y `mkinitramfs`.

CentOS utiliza la utilidad `mkinitrd` para crear el disco inicial de RAM. A este comando se le pasa un nombre para el fichero, y la versión del kernel.

```
# mkinitrd /boot/initrd-2.6.35.4 2.6.35.4
```

y genera el fichero `/boot/initrd-2.6.35.4.img` a partir de los módulos del kernel 2.6.35.4.

5. Preparar un paquete de kernel

Si se mantienen muchos sistemas Linux, quizás desée crear un paquete RPM para simplificar la instalación de su kernel. Para ello necesitará tener instaladas las herramientas `rpm-pkg`, `binrmp-pkg` o similares, y crearlo desde su directorio de kernel de Linux. El paquete creado de esta manera será análogo al que pueda obtener de su distribuidor, y podrá ser utilizado en los sistemas que compartan la misma arquitectura y hardware que aquél desde el que fue creado.

6. Añadir el kernel al arranque de GRUB

Con el nuevo kernel construido y todos sus ficheros en su sitio, se debe actualizar el fichero de configuración de GRUB para cargar el nuevo kernel en el momento de inicio, o bien para poder seleccionarlo en el arranque.

Aquí puede consultar las instrucciones para compilar un núcleo personalizado en CentOs.

Manejo de los módulos de kernel en el momento de ejecución

Si bien en un principio el kernel de Linux estaba formado por un solo fichero que se cargaba en memoria al inicio, al ir creciendo en funcionalidades (y tamaño), se hace más eficiente extraer la mayor parte de los drivers a módulos externos que ayuden a mantener "pequeño" el fichero de kernel, y a hacerlo compatible con más sistemas, permitiendo a cada usuario seleccionar sus propios drivers. En este apartado veremos cómo manejar los módulos de kernel.

Obtener información acerca del kernel

Contamos con varios comandos para obtener información acerca del kernel y de los módulos del kernel:

uname -a

El comando **`uname -a`** nos da información acerca del kernel (nombre del fichero, versión, etc) y de la máquina (nombre DNS, tipo de CPU, arquitectura, etc). Estos datos son importantes a la hora de conocer la compatibilidad de nuestro sistema con módulos de kernel desarrollados por terceros, y con cierto software.

lsmod

El comando **`lsmod`** proporciona un listado de los módulos de kernel actualmente cargados en el sistema. También se indica qué otros módulos o procesos están haciendo uso de cada módulo listado en ese momento. Este comando **no** da información acerca de drivers que se hayan compilado directamente en el kernel.

modinfo

Con el comando modinfo se puede obtener información más detallada de cada uno de los módulos listados en lsmod. La información que se obtiene depende de cada módulo.

Cargar nuevos módulos de kernel

Existen dos programas para cargar módulos de kernel: `insmod` y `modprobe`.

insmod

`insmod` permite la inserción únicamente de un módulo simultáneamente, por lo cuál es necesario que todos los módulos de los que dependa el que queramos insertar estén cargados previamente. En caso contrario fallará. Su uso es muy sencillo, por ejemplo:

```
# insmod kernel/drivers/net/wireless/airo.ko
```

Se requiere la **ruta completa** al fichero. El comando puede llevar alguna opción, que en la mayoría dependerá del propio módulo, por lo que deberá consultar la documentación del módulo. Más tarde veremos cómo configurar estas opciones desde un fichero de configuración, para modificarlas en un módulo ya cargado.

modprobe

Con `modprobe`, se puede especificar el módulo por su nombre, en lugar de por la ruta a su fichero. Además, carga los módulos dependientes automáticamente. Para ello utiliza ficheros de configuración:

```
# modprobe airo
```

Descargar módulos de kernel

Cuando un módulo está cargado en el kernel y no es utilizado, consume un pequeño espacio de memoria.

lsmod

El comando `lsmod` nos muestra cuánta memoria consume cada módulo, y puede ayudarnos a decidir si eliminar el módulo merece o no la pena. En muchos casos la razón para eliminar un módulo es la de cargar una versión nueva del mismo, no tanto por liberar memoria.

rmmod

El comando "opuesto" a `lsmod` es `rmmod`, pero en este caso es suficiente con pasarle el nombre del módulo como parámetro.

```
# rmmod cdrom
```

rmmod -w

Si se trata de descargar un módulo que o bien depende de otros que están aún instalados, o bien está siendo usado por algún proceso, `rmmod` devolverá un error. Con la opción `-w` podemos esperar a que el módulo que está siendo usado sea liberado y eliminarlo entonces (aunque si acaba de ser usado, tal vez esté teniendo más uso del que creíamos...). Si el error se produce debido a dependencias, los módulos dependientes aparecerán listados, y deberemos ir descargándolos uno a uno antes de descargar el definitivo.

Herramientas de mantenimiento de los módulos de kernel

Los comandos `lsmod`, `modprobe` y `rmmod` utilizan ficheros de configuración, como hemos mencionado recientemente. Los ficheros principales son **`/etc/modules.conf`** y `/etc/modprobe.conf`. Estos ficheros se pueden editar para pasarle parámetros al módulo. Como ya mencionamos, las opciones varían según módulos, y deben de consultarse en la documentación del mismo. Por ejemplo, editando el fichero `/etc/modprobe.conf` y añadiendo la línea

```
options sisfb mode=1280x10324 rate=75
```

indicamos al driver de tarjeta gráfica la resolución y tasa de refresco por defecto, a la que se desea configurar ese dispositivo. A continuación, utilizaríamos el comando `depmod` para descargar el módulo, y una vez recargado, se aplicarían los nuevos valores. También en cada reinicio del sistema ya se tomarían esos nuevos valores.

Las dependencias entre módulos se manejan desde el fichero denominado `modules.dep`, que se encuentra en el directorio principal que contiene los módulos del kernel, `/lib/modules/version`. Para manipular este fichero se utiliza el comando `depmod`. Por ejemplo, sin opciones y ejecutado como `root`, regenerará el fichero modules.dep . Esto también se hace automáticamente al instalar nuevos módulos.

En resumen...

En este módulo hemos hablado sobre el kernel de Linux y los módulos del kernel, haciendo hincapié en su compilación, instalación y mantenimiento.

Glosario

Glosario de términos de interés para el curso

- **Bootloader**

  Un gestor de arranque (en inglés «bootloader») es un programa sencillo que no tiene la totalidad de las funcionalidades de un sistema operativo, y que está diseñado exclusivamente para preparar todo lo que necesita el sistema operativo para funcionar. Normalmente se utilizan los cargadores de arranque multietapas, en los que varios programas pequeños se suman los unos a los otros, hasta que el último de ellos carga el sistema operativo.

- **Chipset**

  Un chipset es el conjunto de circuitos integrados diseñados con base a la arquitectura de un procesador para permitir que ese tipo de procesadores funcionen en una placa base específica. Sirven de puente de comunicación con el resto de componentes de la placa.

- **Distribuciones**

  Una distribución Linux (coloquialmente llamada distro) es una distribución de software basada en el núcleo Linux que incluye determinados paquetes de software para satisfacer las necesidades de un grupo específico de usuarios, dando así origen a ediciones domésticas, empresariales y para servidores. Por lo general están compuestas, total o mayoritariamente, de software libre, aunque a menudo incorporan aplicaciones o controladores propietarios. Además del núcleo Linux, las distribuciones incluyen habitualmente las bibliotecas y herramientas del proyecto GNU y el sistema de ventanas X Window System. Existen distribuciones que están soportadas comercialmente, como Fedora (Red Hat), openSUSE (Novell), Ubuntu (Canonical Ltd.), Mandriva, y distribuciones mantenidas por la comunidad como Debian y Gentoo.

- **Gestor de arranque**

  En gestor de arranque (en inglés «bootloader») es un programa sencillo que no tiene la totalidad de las funcionalidades de un sistema operativo, y que está diseñado exclusivamente para preparar todo lo que necesita el sistema operativo para funcionar. Normalmente se utilizan los cargadores de arranque multietapas, en los que varios programas pequeños se suman los unos a los otros, hasta que el último de ellos carga el sistema operativo.

- **GRUB**

  GNU GRUB (GNU GRand Unified Bootloader) es un gestor de arranque múltiple que se usa comúnmente para iniciar uno, dos o más sistemas operativos instalados en un mismo equipo.

- **Inodo**

  Un inodo en un disco (dispositivo en modo bloque) es una estructura que almacena información sobre archivos regulares, directorios, y enlaces simbólicos. El concepto de inodo es particularmente importante para la recuperación de los sistemas de archivos dañados.

- **Kernel**

  El kernel es el software que sirve de puente entre el hardware y el resto de funciones del sistema operativo, que maneja el sistema de ficheros, el acceso a la red y otras tareas de bajo nivel que son críticas para el rendimiento. Está almacenado en un fichero principal y varios módulos.

- **Tarballs**

  El programa tar, es usado para almacenar archivos y directorios en un solo archivo. El fichero resultante tiene la extensión .tar y se denomina coloquialmente "tarball".

Ejercicio: Ejecutar make xconfig

Como ya se vió en la lección, el comando `make xconfig` nos sirve para poder especificar las opciones de compilación del kernel dentro de un entorno gráfico.

Se le pide que configure su equipo CentOs para que se posibilite la ejecución de este comando. Varios módulos deben de estar instalados previamente. Utilice la documentación del curso y su capacidad de análisis y resolución de errores para instalar todos los módulos que su equipo necesita para que al ejecutar el comando `make xconfig` desde `/usr/src/kernels` se visualice la aplicación deseada.